

PHYSICAL TOUCH OF VIRTUAL OBJECTS

ROKAS ANDRIJAUSKAS 4129184

Masters Project in Engineering (Robot Systems)
Supervisor: Xiaofeng Xiong

June 2025



The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Character count: 29021

Abstract

Many companies are struggling to combine Robotics and Machine Learning . .

Contents

Contents	ii
1 Introduction	1
1.1 Background	1
1.2 Problem statement	1
1.3 Objective	2
1.4 Research question	2
2 Exoskeleton	3
2.1 Background	3
2.2 Analysis and Design 1	9
2.3 Implementation 1	10
2.4 Analysis and Design 2	18
2.5 Conclusion	18
3 Algorithms	20
3.1 Background	20
3.2 Analysis and Design	20
3.3 Implementation	20
3.4 Conclusion	20
4 Discussion	21
5 Conclusion	22
A Appendix	23
Bibliography	24

1 Introduction

1.1 Background

Recent advancements in computer hardware and software have enabled nearly photorealistic and real-time simulation of virtual worlds. Technologies such as augmented reality (AR) and virtual reality (VR) enhance the immersion of these simulated world by overlaying information over the user's field of view. Since the technology is still in its early stages, there is still a lot of unsolved problems and progress to be made. The immersion into virtual worlds is usually achieved through the use of a headset. Early headsets were really bulky, had low resolution and short battery life. Naturally most of the research was focused on tackling these problems. In the present day we have consumer grade devices like meta quest 3 and apple vision pro that offer excelent battery life, sharp visuals and immersive audio. In the future we could expect this technology to shrink to the size of regular pair of glasses. Resarchers at Meta are already stepping in that direction with ther recently unveiled Orion glasses.

The hardware improvements also made the creation of Artificial Intelligence (AI) possible. It was theorized as early as 1950s by Alan Turing if machines are capable of thinking [23]. In 1959 Arthur Samuel coined the term 'machine learning', but Ii was only in 1990s that hardware and algorithms were sufficiently advanced that machine learning began to gain traction. Due to well-defined environments and rules, games were the first domain to notice the impact of AI. In 1997, Deep Blue [2] managed to beat the world chess champion Garry Kasparov. As computers following Moore's law [18] grew in power, more breakthroughs began to emerge. Currently, in 2025, the AI is capable creating images, videos, music, text, research podcasts and even interactive worlds.

The artificial intelligence also gave rise to the robotics industry. AI is transforming robots from rigid, pre-programmed machines into intelligent systems capable of performing increasingly complex tasks. Robots are often trained in simulated environments as this allows for faster and safer training without needing to worry about demaging robot or the environment.

1.2 Problem statement

Despite the fact that VR and AR technologies are rapidly improving, there is still a major problem, these technologies usually immerse only two senses: sight

and hearing, but we, as humans, have five: sight, hearing, taste, smell, and touch. This means that there is still a lot of room for improvement as inclusion of additional senses would provide a more immersive VR/AR experience.

1.3 Objective

The thesis aims to address the limitation of VR technology by incorporating the sense of touch into the computer-simulated virtual worlds. There are two objectives, first, following engineering design process, will be focused on developing a portable exoskeleton capable of giving its user an immersive sense of touch in VR. Second, will be dedicated to exploring if the developed exoskeleton could aid in the process of training robots in simulated world.

1.4 Research question

Can a portable exoskeleton that gives its users the ability to touch virtual objects be created in a way that it is a viable solution to aid in the training of the robots in the simulated environments?

2 Exoskeleton

This chapter delves into the research, development and testing of the portable exoskeleton.

2.1 Background

In this section a comparison of physics world simulators and state of the art exoskeletons that attempt to incorporate touch into virtual environments will be made.

Virtual environment

The hard requirements for the choice of the simulator for this thesis are simple, it must have:

1. VR support. Users should be able to interact with the environment.
2. Support of the soft-body physics.

A great to have features for the simulator are the following:

1. Free and open-source, so the research in this thesis could be easily accessible and replicated.
2. Fast, so the development would go smoothly on an affordable computer.
3. Popular, it is easier to fix problems if we can find people in forums who already had them.
4. Easy to learn, it would be great to have more time on research and implementation than learning the engine.
5. Used in robotics applications.

Numerous physics engines have been developed in which virtual environments can be created. Most notable are [5]: Mujoco, Gazebo, CoppeliaSim, Webots, PyBullet and SOFA.

Multi-Joint dynamics with Contact (MuJoCo) [22] is a popular physics engine that required a commercial license up until recently. In 2021 Google Deepmind acquired the simulator and made it freely available for everyone [7]. In 2022, they open-sourced and published it on Github. The MuJoCo is widely used

in research due to its contact stability [25]. State of the art research leverages MuJoCo for training robotic manipulator policies in simulation. It is worth noting that MuJoCo lacks built-in functionality for inverse kinematics or path planning.

Gazebo is another popular robotics simulator that is widely used in research. One of the things that makes it so appealing is support of Robotics Operating System [19] (ROS). ROS houses a expansive set of libraries and tools for building robotics applications. ROS-based software is platform and language agnostic making incorporation of user developed libraries very easy. The working principle that allows for this seamless integration is Publish/Subscribe model. ROS processes called nodes, communicate between each other by publishing messages to topics and receiving messages from subscribed topics allowing interprocess communication. A study has shown that if the robot model in Gazebo was created properly, the code that was used in simulation can be directly reused in real robot without any modifications [21]. Unfortunately the engine does not support soft-body physics natively. There are however attempts to bridge this gap between rigid-body and soft-body physics. One study developed a ROS-Gazebo Toolbox that allows user to bring Articulated Soft Robots in the simulation with satisfactory Sim2Real performance [15]

CoppeliaSim, formerly known as V-REP [20], has an easy learning curve with it's GUI driven design and user- centric feature including forward and inverse kinematics, sensor simulation, motion and path planning. Built around a distributed control architecture, the engine supports scripts written in Lua or Python and plugins written in C/C++ that act as synchronous controllers. This is useful in multi-robot setups. It also has ROS support, but unlike Gazebo, it is secondary to the engine's design. CoppeliaSim supports 5 different physics engines (MuJoCo, Bullet Physics, ODE, Newton and Vortex Dynamics), but the support for soft body and deformable objects is lacking.

Webots [16], started in 1996, is an open-source simulator witten in C++. It has a large collection of sensors, actuator and robot models. The simulator is beginner friendly with polished GUI that makes it easy to build models from scratch. One study comparing robot simulators has shown that Webots was least resource intensive, it the lowest CPU usage compared to CoppeliaSim and Gazebo [1]. These reasons make the simulator appealing for fast prototyping and multi-robot simulation purposes. Despite these great features the simulator lacks soft-body dynamics support.

Bullet is a professional library written in C++. It is videly used in games, visual effects and robotics applications. It can simulate Soft-Body as well as Rigid-Body physics with discrete and continuous collision detection. The PyBullet is a library with Python bindings that interact with the Bullet engine.

Simulator	Force Sensor	Inverse Kinematics	Soft-Body	VR	ROS
MuJoCo	✓	✗	✓	✓	✗
Gazebo	✓	✓	✗	✓	✓
CoppeliaSim	✓	✓	✗	✓	✓
Webots	✓	✗	✗	✓	✓
PyBullet	✓	✓	✓	✓	✗

Table 2.1: Feature Comparison of Popular Physics Simulators for Robotics Applications

This approach merges best of both worlds, the C++ speed and Python’s ease of use. PyBullet also has reinforcement learning, robotics simulation and VR support.

From the examined physics engines only MuJoCo and PyBullet support deformable objects and virtual reality. PyBullet was chosen as the physics engine for this thesis as it has a well documented VR section and broader VR support. Teleoperation with MuJoCo is only available through the use of MuJoCo HAPTIX [13] whereas PyBullet support a variety of VR controllers.

Exoskeletons

Skin is the largest human organ. The density of touch receptors varies throughout different skin regions. The palms of the hand, especially fingertips are one of the most touch sensitive parts of the body. This makes sense as hands are the primary way we interact with the environment. Creating an exoskeleton that covers the whole body is too expensive and too big of a task to take up on alone, so the focus of this thesis will be to create an exoskeleton suited for the hand only. There have already been multiple attempts to create an exoskeleton that allows user to feel the virtual objects. Some have even become commercially available.

Types of feedback

There are many types of feedback a simple touch can provide. They will be discussed in this section.

Transient contact and normal force provide important information about perceived object stiffness. A study has shown that this type of contact can be modeled easily without complex mechatronics / control systems. They have achieved this using transient vibration and visuo-haptic illusion [4].

Lateral stretch of the skin provides information about the weight of the object and the stick-slip condition gives us an insight on when we are about to drop the object. That allows us to react quickly by gripping the object tighter or

catching it mid-air. Different approaches and designs have been investigated to achieve this [17][3]

Skin is also capable of recognizing the texture of objects. A study has shown that it is feasible to simulate texture perception with vibratory-only feedback by leveraging spatial information from skin interactions with textures [24]

Temperature feedback can also provide important information about the type of material we are in contact with. Different heat flows and temperature gradients are experienced when touching objects due to the different thermal conductivity coefficients between materials. A wearable device has been proposed for the VR setting. It contains a film with high thermal conductivity controller by Pielteled element attached to water-cooling heatsink [10].

As we can see, there are systems that propose how to achieve the different types of feedbacks, but combining multiple types remains a difficult engineering challenge.

Tracking

There are two types of tracking. Global position tracking that provides insight on whereabouts of your hand in the world space and local position tracking that provides information on the shape of your hand, where each of your finger is and how it is positioned. There are existing tracking technology that can simply be attached to the exoskeleton to provide global position tracking.

Vive tracker, the first commercially available VR tracker developed by HTC enable object and body tracking in virtual reality environments. These small, lightweight and fully wireless trackers can be attached to body parts like hands, feet, or waists to track their position and orientation in 3D space.

Optitrack camera system can track position of reflective objects with very high precision. Special reflective markers are put on the object of interest. These markers reflect infrared light emitted by Optitrack camera system. This allows cameras to determine marker position in space.

Since most VR systems you buy come along with VR controllers, to save some costs, some exoskeletons provide global position tracking by simply attaching the controller to the exoskeleton. However, this approach does have the disadvantage of unnecessary components increasing the weight of the exoskeleton. Local position tracking can be achieved in several ways as well.

Valve index knuckles use a proximity sensor to detect if finger is touching the controller or not. This results in only 5 Degrees of Freedom (DOF) where each finger is in a binary state, either extended or flexed.

Data gloves can provide continuous finger state tracking. Some data gloves [8] achieve that using flex capacitive stretch sensors. These sensors work due to their physical properties that resistance changes based on mechanical deformation. This fact means that they wear and thus have a limited life cycle. Modern flex capacitive stretch sensors achieve life cycle of >1 million cycles. Some data gloves [14] achieve local position tracking using multiple IMUs. This has an advantage of improved life cycle, but these type of gloves do experience “data drifting” which requires frequent re-calibration. Moreover the reliability of IMU sensor is influenced by metal and magnetic fields nearby making them difficult to use with motor based exoskeletons.

Vision based solution like leap motion provide the highest number of degrees of freedom. They do only provide the tracking if the hands are in camera’s field of view and may loose accuracy the more hands are obstructed. The reliability of these systems is yet to be tested when wearing an exoskeleton.

Prototypes and products

Dexta Robotics have been researching human-machine-interactions for virtual and mixed reality since 2014. They believe that user interface define user experience, the way motion controllers took over console controllers in VR space, the next big step could be for glove controllers replace motion controllers to achieve more intuitive button free hands-like experience. Their latest model Dexmo Force Feedback Gloves are available purchase privately for enterprises or research institutions, to be used in: VR arcade, training simulation, entertainment, colleges and other scientific research institutions. The consumer edition is still under development. They’re design relies on rigid kinematic links actuated with a small gear motors, which were designed and manufactured in-house to provide the small form factor. They are capable of providing 0.5N of torque each. The system is fully wireless with a battery life of up to 6H. The whole system weights under 300g. These gloves use rotational sensors to achieve 11 DOF local position tracking.

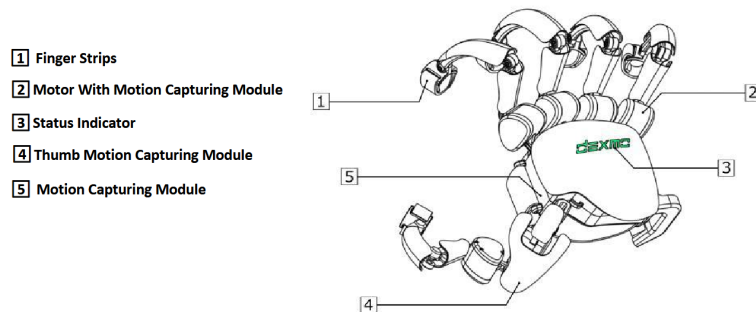


Figure 2.1: Design of Dexmo Glove

Sensaglove Nova 2 uses magnetic friction brakes that transfer the force to the fingertips through mechanical wires. Each brake can deliver up to 20N of force. Additionally vibrotactile feedback is provided for the fingertips using voice coil actuators. The local position tracking is provided by combining sensor-based finger tracking with computer vision hand tracking algorithms.

DextrES [11] research paper proposes a more innovative yet experimental approach. They rely on flexible metal strips to generate an electrically-controlled friction force. Such design is much more compact and light as it does not rely on motors. The haptic glove that weighs only 8g is capable of generating up to 20N of holding force for each finger.

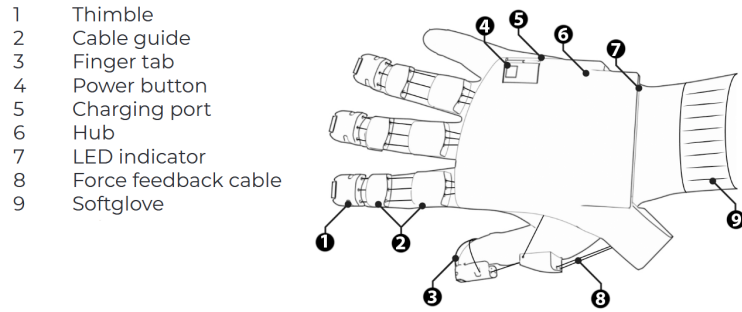


Figure 2.2: Design of Sensaglove Nova 2

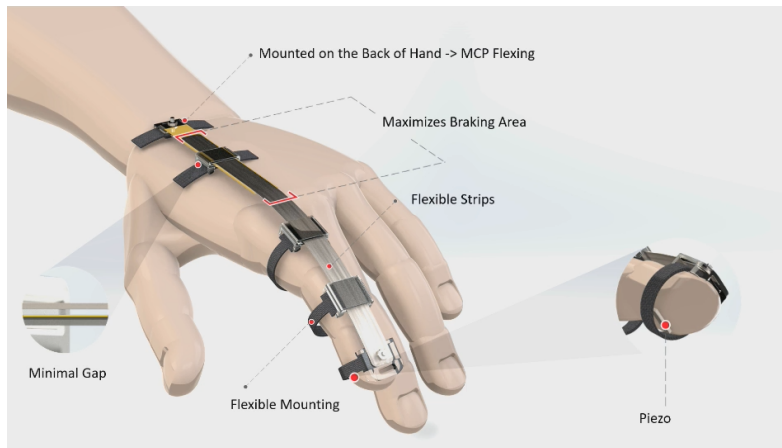


Figure 2.3: Design of DextrES

Lastly, there has been proposed a gripper like exoskeleton for the purpose of learning from demonstration [6]. With the use of displacement and IMU

sensors Fig 2.4, it is capable of collecting position, orientation and gripper displacement information. It has no force feedback as it is designed to grip real objects.

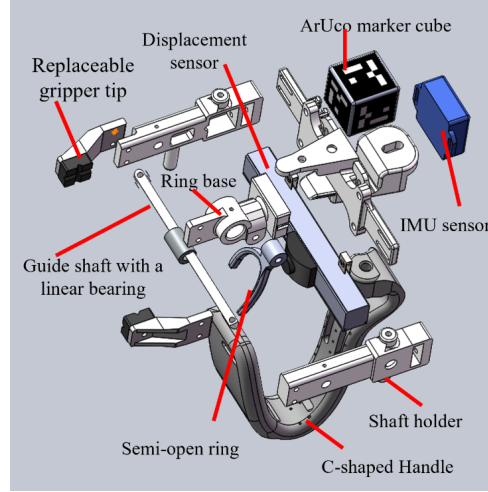


Figure 2.4: Design of gripper like exoskeleton

2.2 Analysis and Design 1

A design using magnetic friction brakes is not an ideal solution as it provides feedback primarily during the squeezing of virtual object, during releasing the brakes are disengaged and no resistance is felt. This may be a good solution for VR simulations consisting of only rigid objects, but since this thesis is focused on soft objects an exoskeleton with servo motors will be used. The purpose of the first design is to get a good idea of the scope of the project, make a working prototype quickly and detect the possible issues with the project early on. The prototype, which from now on will be referred to as RoboGripVR V1, consists of two Dynamixel XM540-W270-R servo motors connected in daisy-chain using single line of RS-485 communication. The Dynamixel XM540-W270-R is a high-performance and precision servo motor from Robotis, this motor can provide 10.6 Nm of torque at 12V. The SMPS2Dynamixel is a power converter for the motors and U2D2 is a USB communication converter that enables to control and operate DYNAMIXEL motors using a personal computer (PC) Fig 2.5.

This design is not in the glove form, so it does not have sensors for global position tracking. The squeezing motion can be performed using thumb and index finger.

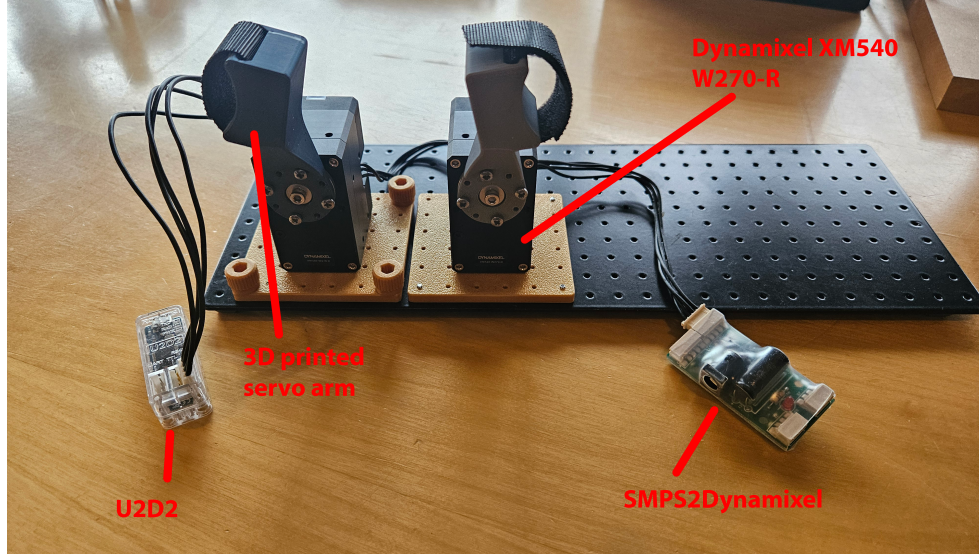


Figure 2.5: Design of RoboGripVR V1

2.3 Implementation 1

The PyBullet environment for testing this design consists of a table, soft ball and a robotiq 140 gripper Fig 2.6. The table and gripper were imported using a Unified Robot Description Format (URDF) an XML-based file format used to describe the physical and kinematic properties of a robot. The soft ball was created using open source 3D modeling software - Blender. The blender file exported as .obj provides only the surface mech. To obtain volumetric mesh needed to simulate the soft body, open source software - GMSH was used.



Figure 2.6: PyBullet environment for testing RoboGripVR V1

The gripper has stroke range of 0 to 0.127 units in the simulation. It is controlled by a single motor using position control. A function that controls the robotiq gripper based on distance instead of angle was created by linearly spacing 100 points of gripper's stroke range, measuring the angle of the motor and

the distance between the fingerpads in simulation and fitting this data using a polynomial Fig 2.7.

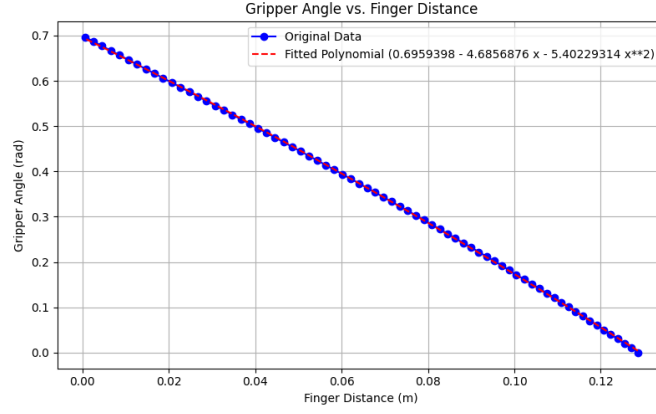


Figure 2.7: Polynomial fitting of data between gripper's motor angle and distance between finger pads

The Dynamixel motors are controlled with the python files provided in their SDK. The distance between finger pads can be calculated using simple trigonometry Eq 2.1, Fig 2.8. This distance obtained from the motors is scaled and applied to the simulation every frame.

$$\begin{aligned}
 p_1 &= r_1 \cdot \cos(a_1) \\
 p_2 &= r_2 \cdot \cos(a_2) \\
 d_{total} &= -p_1 + d_0 + p_2
 \end{aligned} \tag{2.1}$$

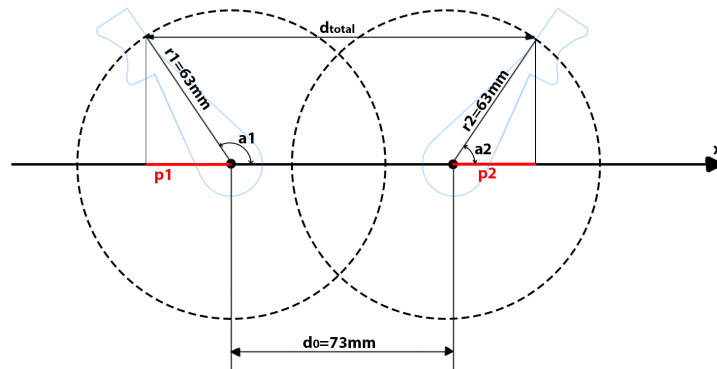


Figure 2.8: Diagram of distance calculation for RoboGripVR V1

The PyBullet's Neo-Hookean model was used for soft-body simulation. While linear models like Hooker's Law can be used to model small strains, materials

that undergo large deformations need to use a hyperelastic model that account for their nonlinearities. Neo-Hookean is a simple form of hyperelastic model. It is characterized by strain energy density function, which defines the energy stored in the material due to deformation. The Neo-Hookean model decouples this energy into two parts, deviatoric that handles shear deformations and volumetric part that deals with volume changes. The model is configured by material dependant Lamé parameters that describe the mechanical behavior of isotropic solids. The first Lamé parameter λ is related to the volumetric changes and the second Lamé parameter (shear modulus) μ is used to describe the material's resistance to shear deformation. Both of these parameters can be calculated [9] from material properties E (Young's modulus) and ν (Poisson's ratio) Eq 2.2, 2.3.

$$\mu = \frac{E}{2(1 + \nu)} \quad (2.2)$$

$$\lambda = \frac{E\nu}{(1 + \nu)(1 - 2\nu)} \quad (2.3)$$

A study [12] has shown that it is possible predict these parameters using data driven model to accurately simulate real world soft models in PyBullet. It also mentions that soft-bodies tend to become unstable (implode or explode) when Lamé parameters are set too low or to high. Another reason for soft-body instability is that the simulation is run in discrete time steps, the lack of fidelity can cause numerical issues during sudden application of large forces. Some solvers use adaptive time stepping to overcome this issue, but such feature is unavailable in PyBullet. It's a common practice to set PyBullet's time step to 1/250 of a second, but this value turned out to have poor stability during the experiments. A value of 1/1000 was used as it yielded much better stability. Another factor that influenced stability was the mesh itself. Multiple balls were created using icosphere mesh with varying subdivision setting Fig 2.9. Increasing subdivisions increases the resolution of the mesh, making the ball appear smoother, but it also increases the vertex count potentially making soft body simulation unstable or slower. A ball with 2 subdivisions was used as it yielded the best performance and stability.

The normal forces between soft ball and gripper pads are obtained using PyBullet's function `getContactPoints`. The reaction forces R_l and R_r in PyBullet are assumed to be distributed symetrically and can be represented with total resultant force as a sum of individual forces at the centroid of the contact area. In order to achieve the force feedback in the exoskeleton, the reaction forces from simulation are converted to motor torques Eq 2.4

$$\begin{aligned} \tau_l &= R_l d_l \\ \tau_r &= R_r d_r \end{aligned} \quad (2.4)$$

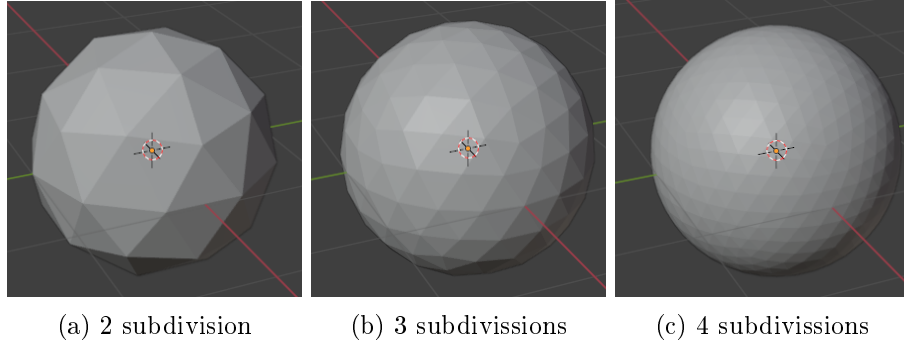


Figure 2.9: Varying subdivision setting

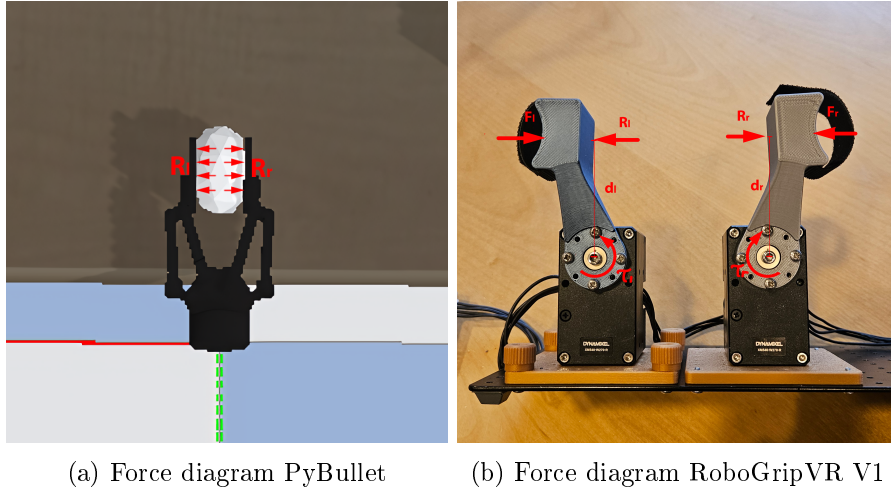


Figure 2.10: Force diagrams

XM540-W270-T/R are smart servo motors; they have several control modes available: Current Control Mode, Velocity Control Mode, Position Control Mode ($0\ 360\ [^\circ]$) and PWM Control Mode (Voltage Control Mode). The current control mode and PWM control mode could both be suitable in this scenario, so they will be tested and comparison between them will be made.

PWM control

The Pulse Width Modulation (PWM) is a low level motor control mode and is the final step in all of the control modes. It can give fine grain control over the motors if the available control modes are not satisfactory. Additionally, not all motors provide multiple control modes, it is common for direct current (DC) motors to only use PWM control as a means to change the speed of the motor. PWM signal is a square wave with the signal either being on (set to the maximum voltage) or off (zero voltage). The average voltage delivered to

the motor depends on the duty cycle, a ratio of the "on" time to the total period of the signal. For the motor used $PWM \in \mathbb{Z} \cap [-885, 885]$, where 0 would be duty cycle 0%, 885 duty cycle 100% in counter clock wise (CCW) direction and -885 duty cycle 100% in clock wise (CW) direction.

$$V_{out} = V_{max} \cdot DutyCycle \quad (2.5)$$

The motor torque holds a linear relationship to the supplied current Eq 2.6, where k_t is torque constant. From the XM540-W270-T/R motor's datasheet, we can see that stall torque is 10.6 [Nm] (at 12.0 [V], 4.4 [A]). Using this information the torque constant can be calculated: $k_t = \tau/I = 10.6/4.4 = 2.409$

$$\tau = k_t \cdot I \quad (2.6)$$

The motor can measure the supplied current, thus the goal of PWM control will be to reach and maintain a goal current instead of torque as the value can be converted back to the torque later on. The PID controller diagram can be seen in Fig 2.11, the motor's servo arm is placed against a fixed support, once the PWM signal is set, the reaction force from the fixed support generate resistance that is measured by the motor in form of current.

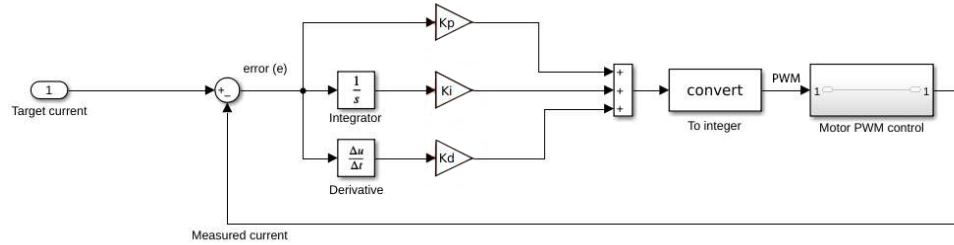


Figure 2.11: Diagram of PID controller

The K_p , K_i and K_d gains need to be tuned. When the transfer function is known, analytical methods such as root locus could be used to tune a PID controller to meet performance requirements. However, for model-free systems, manual tuning is commonly employed, or heuristic approaches like Ziegler–Nichols can be used to obtain initial controller parameters. Manual tuning was employed, but it proved to be difficult and time consuming. It was also very easy to get stuck in a local minima as drastic changes in gains require to restart tuning process. During this trial and error phase an interesting idea occurred to me. By employing evolutionary algorithm (EA), variety of gains could be tested autonomously based on desired fitness function. When the

ball in simulation is squeezed the reaction forces and thus the target currents will be changing rapidly. Intuitively, a good fitness function should encompass rapid changes. For this reason a target current is chosen as a step function Eq 2.7 and the fitness of the individuals will be a sum of absolute errors, meaning the best performing members of the population will be able to reach the target signal with minimal rise time and steady state error.

$$f(\text{iteration}) = \begin{cases} 0.2 & \text{if } 600 < \text{iteration} < 1200 \\ 0.1 & \text{otherwise, where iteration} \leq 1800 \end{cases} \quad (2.7)$$

A population of size 150 was initialized. Each member was assigned random P, I, and D values in ranges $[0, 1024]$, $[0, 512]$, and $[0, 64]$, respectively. The algorithm was trained for 20 generations with a mutation rate of 0.1. The top-performing member of the population has the PID values $K_p = 28.809$, $K_I = 503.211$, and $K_D = 0.191$ with the fitness value of 2.295. This is a significant improvement as the fitness for a manually tuned controller is 8.824.

After the training the PID controller was tested against a more varied target function that includes triangle and sinusoidal waves Fig 2.12. The controller is able to follow the target quite well, the fitness for this function is 54.8. On average in the testing function the present current is delayed by 17 iterations, which equates to 131.059ms with average iteration time being 7.709ms

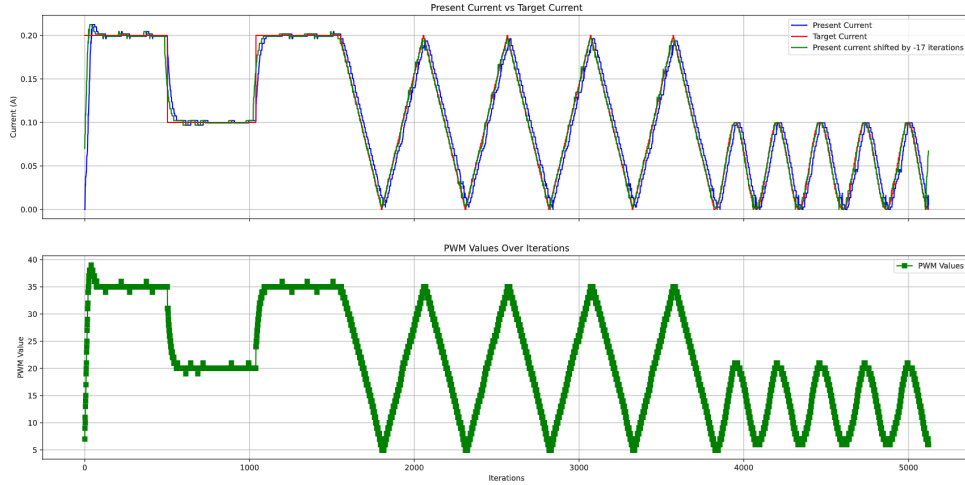
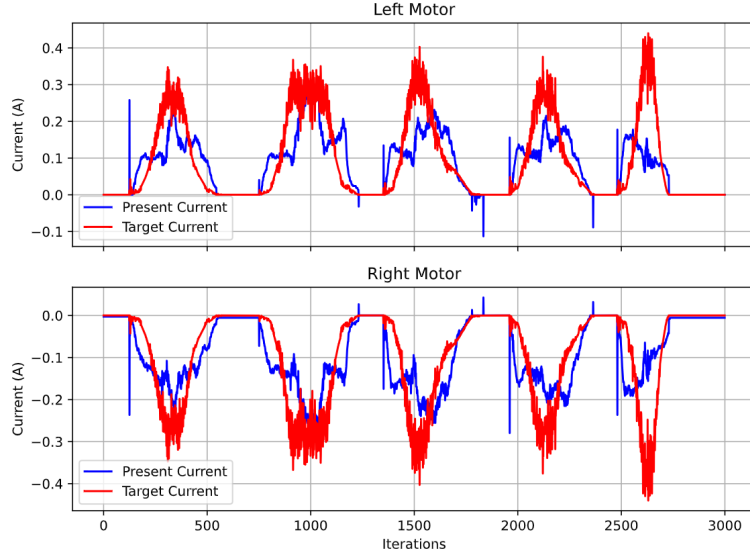


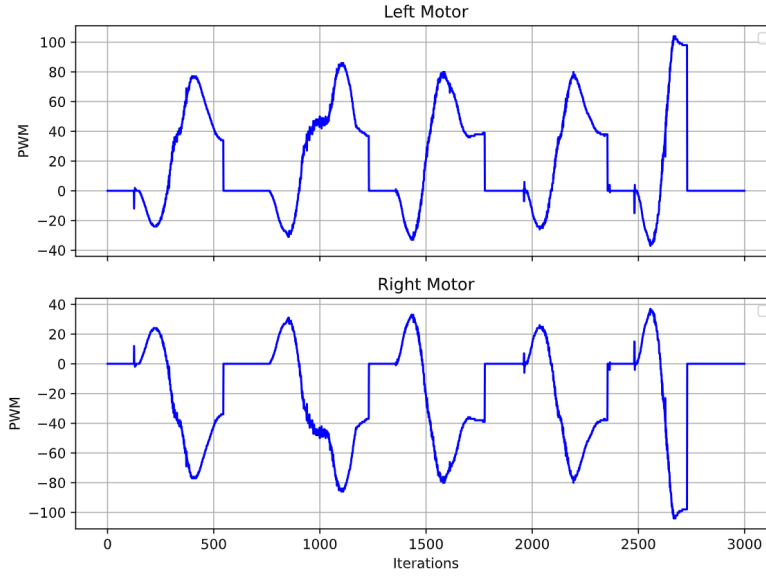
Figure 2.12: Function for testing trained PID values

The exoskeleton was also tested by squeezing the virtual ball. The sensation subjectively feels convincing, but looking at the Fig 2.13 it is evident that the significant delay in reaching target current results in a discrepancy between the forces according to the simulation and forces that are felt. Note that the

controller is disengaged when the target current is zero, otherwise the PWM controller generates resistance instead of being in freewheeling condition.



(a) Current



(b) PWM

Figure 2.13: Squeezing the virtual ball 5 times using PWM control

Current control

This control mode allows the current to be controlled directly. Internally it works similarly to the controller we have implemented in the section above.

As seen in Fig 2.14 when tested, this controller has a much better performance compare to out implemented PWM control, with fitness (sum of absolute errors) being 18.823 and reaching target current in less than 8ms (one iteration, exact time is unknown as this is as fast as we can measure). This performance can be attributed to Dynamixel having well-tuned PID loops optimized for their hardware and low latency feedback due to motor performing controlling internally without the need of slow communication and signal conversion to the PC.

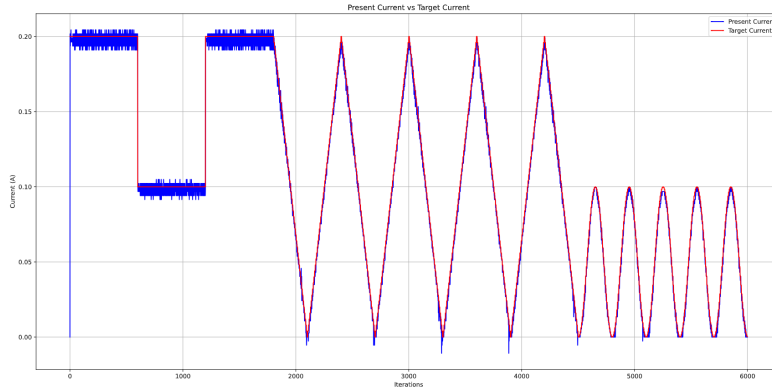


Figure 2.14: Current control test

When squeezing the virtual ball (Fig 2.15) we can see that the current is followed closely, but subjectively the experience does not feel smooth, rapid oscillations in target current occur. By taking a closer look to a single squeeze Fig 2.17a we can see how uneven the target current is. This issues could be mitigated by processing the signal using exponential smoothing Eq 2.8. A variety of different window size values have been tested and plotted 2.16. Increasing the window size provides smoother signal but adds a higher delay. For the controller a windows size of 10 is used as it provides a smoother signal without much delay 2.17b. This controller still does struggle to maintain low target current when suddenly introduced with external forces from the fingers.

$$S_t = \alpha X_t + (1 - \alpha)S_{t-1}, \quad \text{where } \alpha = \frac{2}{K + 1} \quad (2.8)$$

where:

- S_t is the smoothed value at time t ,
- X_t is the actual value at time t ,
- α is the smoothing factor, calculated using window size K ,
- S_{t-1} is the previous smoothed value.

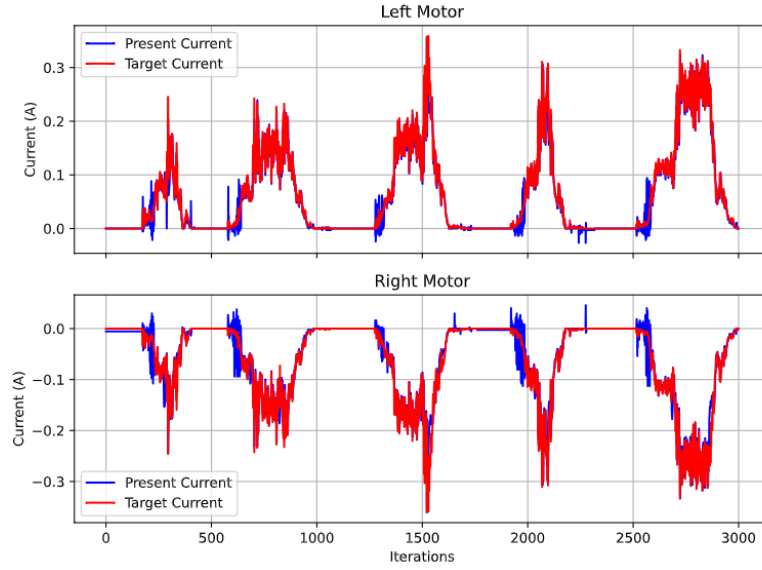


Figure 2.15: Squeezing the virtual ball 5 times using Current control

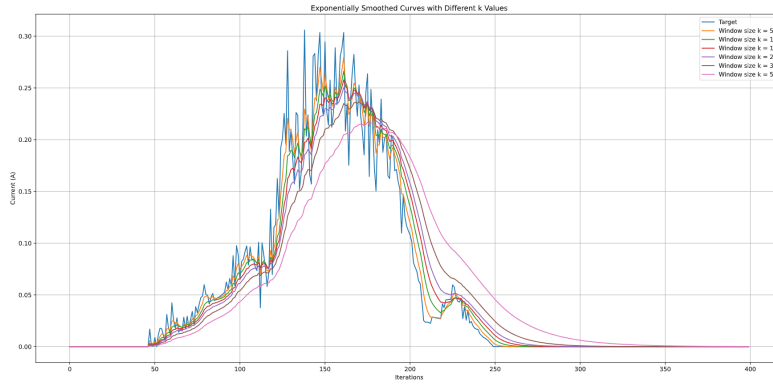
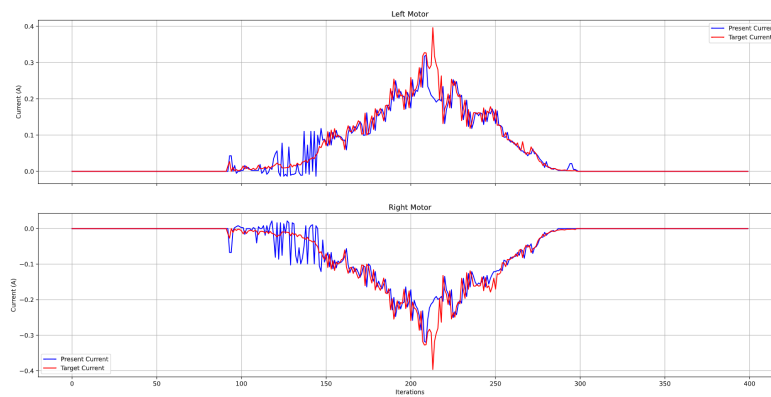


Figure 2.16: Exponential smoothing with different window sizes

2.4 Analysis and Design 2

The goal of this design was to make it suitable to be worn on a hand. It was inspired by the design of the gripper like exoskeleton and the force feedback as provided by Dexmo glove.

2.5 Conclusion



(a) No target current smoothing

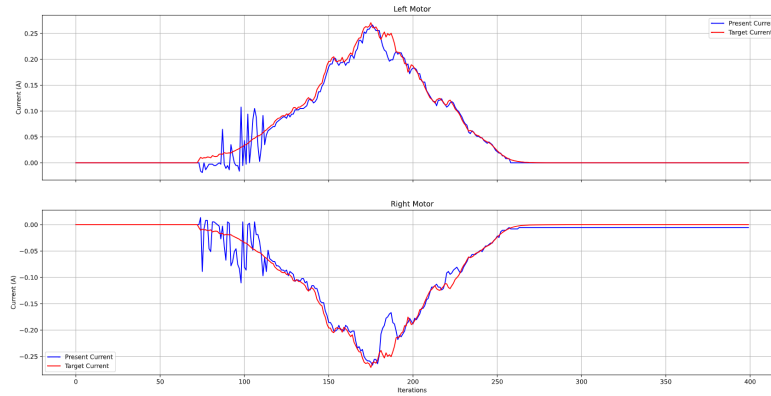
(b) Target current exponential smoothing with $k=10$

Figure 2.17: Comparison of smoothing and no smoothing in target current

3 Algorithms

3.1 Background

In this section we will recall how the...

3.2 Analysis and Design

3.3 Implementation

3.4 Conclusion

4 Discussion

Summing up all the main parts 1 to n conclusions.

5 Conclusion

Overall our project was a success.

A Appendix

We include here the API documentation for our library.

Bibliography

- [1] Angel Ayala, Francisco Cruz, Diego Campos, Rodrigo Rubio, Bruno J. T. Fernandes, and Richard Dazeley. A comparison of humanoid robot simulators: A quantitative approach. *CoRR*, abs/2008.04627, 2020.
- [2] Murray Campbell, A. Joseph Hoane, and Feng hsiung Hsu. Deep blue. *Artificial Intelligence*, 134(1):57–83, 2002.
- [3] Inrak Choi, Heather Culbertson, Mark R. Miller, Alex Olwal, and Sean Follmer. Grability: A wearable haptic interface for simulating weight and grasping in virtual reality. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, UIST ’17, page 119–130, New York, NY, USA, 2017. Association for Computing Machinery.
- [4] Inrak Choi, Yiwei Zhao, Eric J. Gonzalez, and Sean Follmer. Augmenting perceived softness of haptic proxy objects through transient vibration and visuo-haptic illusion in virtual reality. *IEEE Transactions on Visualization and Computer Graphics*, 27(12):4387–4400, 2021.
- [5] Jack Collins, Shelvin Chand, Anthony Vanderkop, and Gerard Howard. A review of physics simulators for robotic applications. *IEEE Access*, PP:1–1, 03 2021.
- [6] Hengtai Dai, Zhenyu Lu, Mengyuan He, and Chenguang Yang. A gripper-like exoskeleton design for robot grasping demonstration. In *Actuators*, volume 12, page 39. MDPI, 2023.
- [7] DeepMind. Open-sourcing mujoco. <https://deepmind.google/discover/blog/open-sourcing-mujoco/>, May 2022.
- [8] Liufeng Fan, Zhan Zhang, Biao Zhu, Decheng Zuo, Xintong Yu, and Yiwei Wang. Smart-data-glove-based gesture recognition for amphibious communication. *Micromachines*, 14(11), 2023.
- [9] FEBio. Febio user manual - neo-hookean model. <https://help.febio.org/docs/FEBioUser-3-7/UM37-4.1.4.17.html>, 2023.
- [10] X. Guo, Y. Zhang, W. Wei, W. Xu, and D. Wang. Thermaltex: A two-modal tactile display for delivering surface texture and thermal information. In *Haptics: Science, Technology, Applications: 12th International Conference, EuroHaptics 2020, Leiden, The Netherlands, September 6–9, 2020, Proceedings*, page 288–296, Berlin, Heidelberg, 2020. Springer-Verlag.

- [11] Ronan Hinchet, Velko Vechev, Herbert Shea, and Otmar Hilliges. Dextres: Wearable haptic feedback for grasping in vr via a thin form-factor electrostatic brake. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology*, UIST '18, page 901–912, New York, NY, USA, 2018. Association for Computing Machinery.
- [12] Fan-Chien Kao, Zhi-Ren Chen, Chung-Shan Shih, Shao-Huang Lu, and Pei-Chun Lin. Bridging mechanical behavior differences of deformable soft objects in simulation and experiments using a data-driven model. In *2024 IEEE International Conference on Advanced Intelligent Mechatronics (AIM)*, pages 1297–1302, 2024.
- [13] Vikash Kumar and Emanuel Todorov. Mujoco haptix: A virtual reality system for hand manipulation. In *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, pages 657–663, 2015.
- [14] Bor-Shing Lin, I-Jung Lee, Pei-Ying Chiang, Shih-Yuan Huang, and Chih-Wei Peng. A modular data glove system for finger and hand motion capture based on inertial sensors. *Journal of Medical and Biological Engineering*, 39, 06 2018.
- [15] Riccardo Mengacci, Giacomo Zambella, Giorgio Grioli, Domenico Caporale, Manuel G. Catalano, and Antonio Bicchi. An open-source ros-gazebo toolbox for simulating robots with compliant actuators. *Frontiers in Robotics and AI*, 8:713083, 2021. Published 2021 Aug 11.
- [16] Olivier Michel. Cyberbotics ltd. webots™: professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):5, 2004.
- [17] Kouta Minamizawa, Souichiro Fukamachi, Hiroyuki Kajimoto, Naoki Kawakami, and Susumu Tachi. Gravity grabber: wearable haptic display to present virtual mass sensation. In *ACM SIGGRAPH 2007 Emerging Technologies*, SIGGRAPH '07, page 8–es, New York, NY, USA, 2007. Association for Computing Machinery.
- [18] Gordon E Moore. Cramming more components onto integrated circuits. *Proceedings of the IEEE*, 86(1):82–85, 1998.
- [19] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Ng. Ros: an open-source robot operating system. volume 3, 01 2009.
- [20] Eric Rohmer, Surya P. N. Singh, and Marc Freese. V-rep: A versatile and scalable robot simulation framework. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1321–1326, 2013.

- [21] Kenta Takaya, Toshinori Asai, Valeri Kroumov, and Florentin Smarandache. Simulation environment for mobile robots testing using ros and gazebo. pages 96–101, 10 2016.
- [22] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *IEEE International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, October 2012.
- [23] A. M. TURING. I.—computing machinery and intelligence. *Mind*, LIX(236):433–460, 10 1950.
- [24] Michael Wiertlewski, José Lozada, and Vincent Hayward. The spatial spectrum of tangential skin displacement can encode tactual texture. *Robotics, IEEE Transactions on*, 27:461 – 472, 07 2011.
- [25] Jaemin Yoon, Bukun Son, and Dongjun Lee. Comparative study of physics engines for robot simulation with mechanical interaction. *Applied Sciences*, 13(2), 2023.